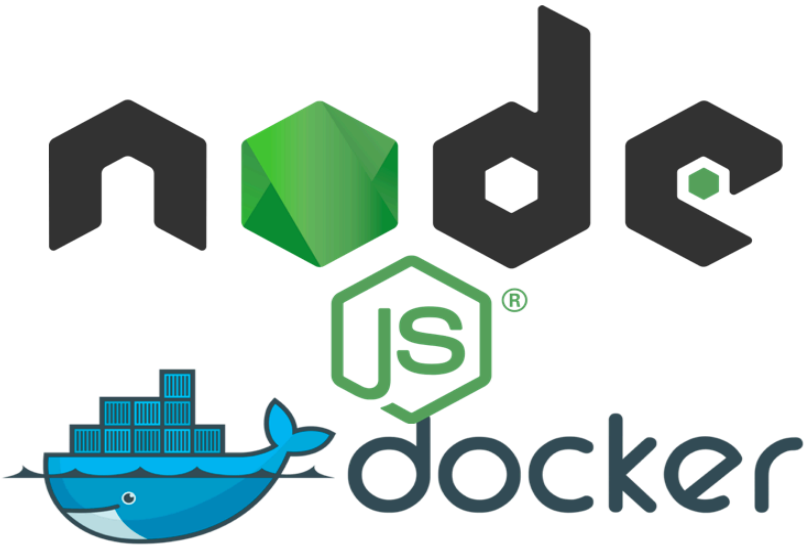
Federico D'Amato 5°Binf V1.0 **22/04/2026 - 23/04/2026**

Container con Codespace e Docker

- Esercizio A - **2**
Container nodejs con GitHub Codespaces **2**
Descrizione del Problema 2
Implementazione 2
Collaudo 2
Procedimento e Risultati 6
Considerazioni Finali 7



DEVELOP & DEPLOY NODE JS WEB APPLICATION IN DOCKER

- Esercizio A -
Container nodejs con GitHub Codespaces

Descrizione del Problema

La richiesta introduce l'uso di Dev Container in GitHub Codespaces per creare ambienti di sviluppo riproducibili. Il problema affrontato consiste nel configurare da zero un container di sviluppo (con Node.js e Docker-in-Docker), avviare un ambiente preconfigurato da un repository esistente, e infine sviluppare ed eseguire una semplice API REST in Node.js/Express all'interno del container, testandone il funzionamento. Si confronta così lo sviluppo diretto nel Dev Container con l'approccio basato su Docker, gettando le basi per la containerizzazione di applicazioni cloud.

Implementazione

- **Laboratorio guidato**

Collaudo

```
MINGW64:/c/Users/feded/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/Progetto/il-mio-lab

feded@LAPTOP-M4VVVMR2 MINGW64 ~
$ cd C:\Users\feded\Desktop\FileSistemi\Esercitazione_6-Codespace&Docker\EsercizioA\Progetto
[1] 1595
bash: cd: C:UsersfededDesktopFileSistemiEsercitazione_6-Codespace: No such file or directory
bash: DockerEsercizioAProgetto: command not found
[1]+  Exit 1                  cd C:\Users\feded\Desktop\FileSistemi\Esercitazione_6-Codespace

feded@LAPTOP-M4VVVMR2 MINGW64 ~
$ cd "C:\Users\feded\Desktop\FileSistemi\Esercitazione_6-Codespace&Docker\EsercizioA\Progetto"

feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/
Progetto (main)
$ mkdir il-mio-lab

feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/
Progetto (main)
$ cd il-mio-lab

feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/
Progetto/il-mio-lab (main)
$ git init
Initialized empty Git repository in C:/Users/feded/Desktop/FileSistemi/Esercitazione_6-Codespace
&Docker/EsercizioA/Progetto/il-mio-lab/.git/
feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Do
cker/EsercizioA/Progetto/il-mio-lab (master)
$ |
```

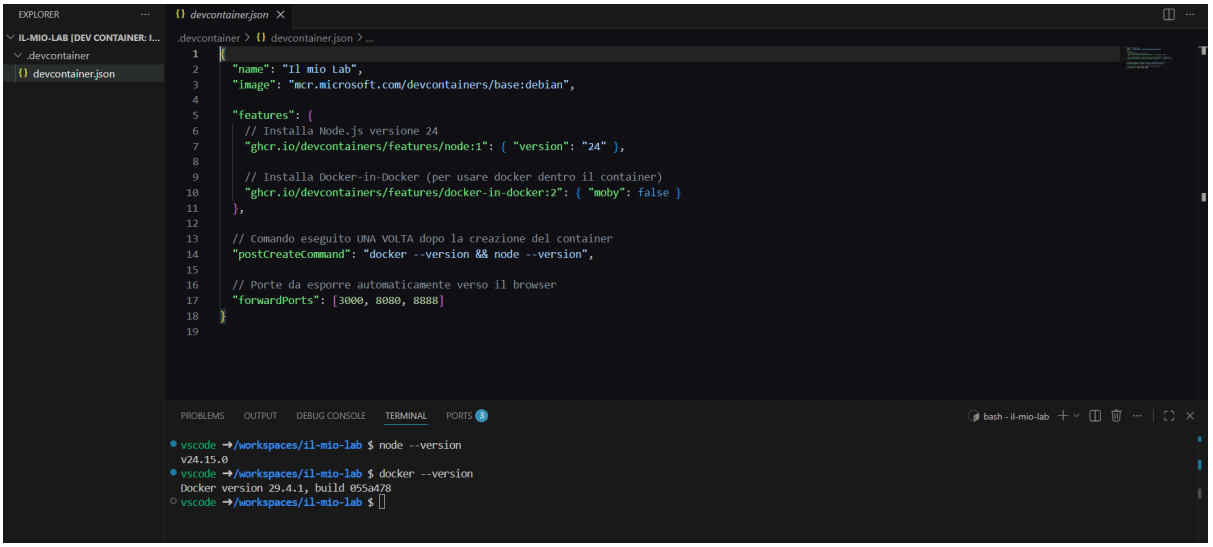
Sul terminale Git Bash, si naviga fino alla cartella di progetto e si esegue mkdir ‘il-mio-lab’ per creare la directory di lavoro. Successivamente si entra nella cartella e si lancia git init per inizializzare un repository Git locale, gettando le basi per il versionamento del codice e della configurazione del Dev Container.

```
MINGW64:/c/Users/feded/Desktop/FileSistemi/Esercitazione_6-Codespace&D...
  GNU nano 8.7                 .devcontainer/devcontainer.json
{
  "name": "Il mio Lab",
  "image": "mcr.microsoft.com/devcontainers/base:debian",

  "features": {
    // Installa Node.js versione 24
    "ghcr.io/devcontainers/features/node:1": { "version": "24" },

    // Installa Docker-in-Docker (per usare docker dentro il container)
    "ghcr.io/devcontainers/features/docker-in-docker:2": { "moby": false }
  },

  // Comando eseguito UNA VOLTA dopo la creazione del container
  "postCreateCommand": "docker --version && node --version",

  // Porte da esporre automaticamente verso il browser
  "forwardPorts": [3000, 8080, 8888]
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

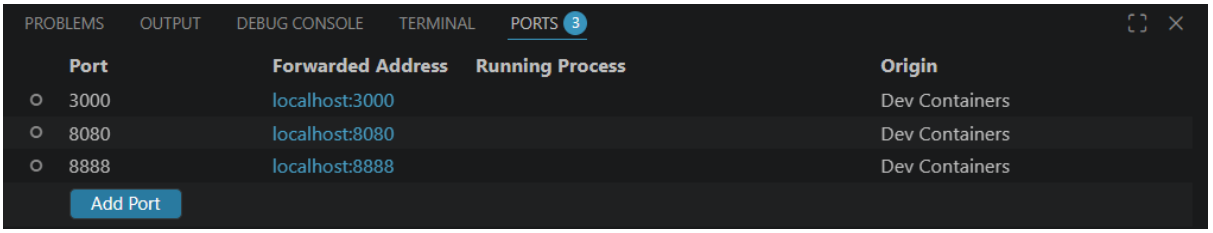
Usando l'editor nano viene creato il file '.devcontainer/devcontainer.json'. Configurazione specifica: nome del container ("Il mio Lab"), immagine base Debian ufficiale, due features (Node.js versione 24 e Docker-in-Docker con moby disabilitato), un comando post-creazione che mostra le versioni di Docker e Node, e le porte 3000, 8080, 8888 da inoltrare automaticamente.

```
feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/
Progetto/il-mio-lab (master)
$ git add .devcontainer/devcontainer.json

feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/
Progetto/il-mio-lab (master)
$ git commit -m "feat: add devcontainer configuration"
[master (root-commit) 5489b4a] feat: add devcontainer configuration
 1 file changed, 18 insertions(+)
 create mode 100644 .devcontainer/devcontainer.json

feded@LAPTOP-M4VVVMR2 MINGW64 ~/Desktop/FileSistemi/Esercitazione_6-Codespace&Docker/EsercizioA/
Progetto/il-mio-lab (master)
$
```

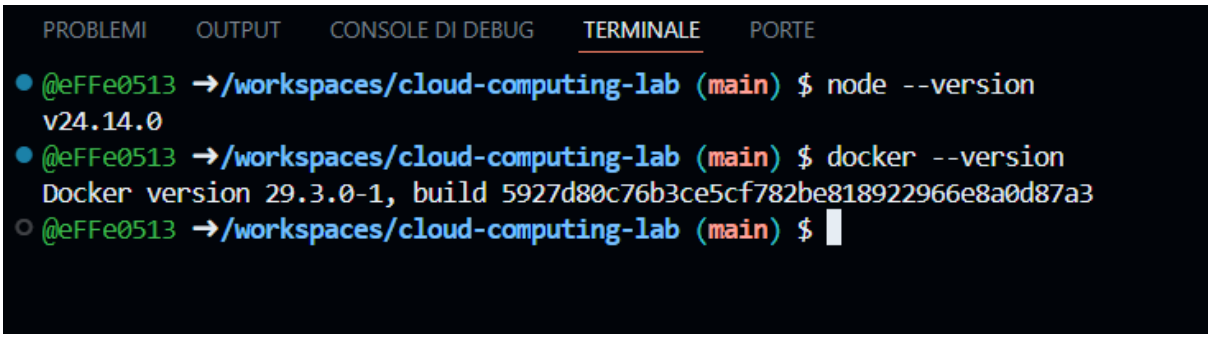
Dopo aver aggiunto il file con git add .devcontainer/devcontainer.json, l'utente esegue git commit -m "feat: add devcontainer configuration". Il commit registra la creazione del file (18 righe inserite) con hash 5489b4a, rendendo persistente la configurazione del Dev Container all'interno del repository locale.



Nell'ambiente VS Code si visualizza il file devcontainer.json aperto. Nel terminale integrato si vede l'output locale: node --version restituisce v24.15.0 e docker --version restituisce Docker version 29.4.1, confermando che Node.js e Docker-in-Docker sono stati installati correttamente nel container.



Dalla sezione PORTS di VS Code si osserva che le tre porte definite (3000, 8080, 8888) sono state automaticamente inoltrate verso localhost dal Dev Container. Ogni porta è associata all'origine Dev Containers e risulta pronta per essere utilizzata da eventuali applicazioni web o API in esecuzione all'interno del container, senza bisogno di configurazioni aggiuntive.
(È opportuno specificare come la prima parte dell'esercitazione sia stata eseguita in locale, mentre la successiva all'interno dell'ambiente Github Codespaces. Ciò ha permesso di testare sia una modalità di programmazione locale che remota).



Nel terminale del Codespace aperto sul repository cloud-computing-lab, l'utente lancia node --version e docker --version. I risultati mostrano Node.js v24.14.0 e Docker 29.3.0-1, confermando che l'ambiente di sviluppo containerizzato è già configurato correttamente con gli strumenti necessari, senza bisogno di installazioni aggiuntive.

```
● @eFFe0513 ➜/workspaces/cloud-computing-lab (main) $ mkdir nodejs-app
● @eFFe0513 ➜/workspaces/cloud-computing-lab (main) $ cd nodejs-app
● @eFFe0513 ➜/workspaces/cloud-computing-lab/nodejs-app (main) $ npm init -y
Wrote to /workspaces/cloud-computing-lab/nodejs-app/package.json:

{
  "name": "nodejs-app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Il prompt del terminale indica che l'utente si trova all'interno della cartella /workspaces/cloud-computing-lab (il repository clonato automaticamente nel Codespace). L'immagine mostra visivamente il percorso completo, evidenziando che si sta operando nella root del progetto, pronti per spostarsi nella sotto-cartella nodejs-app dove verrà scritta l'applicazione.

```
● @eFFe0513 ➜/workspaces/cloud-computing-lab/nodejs-app (main) $ npm install express

  added 65 packages, and audited 66 packages in 3s

  22 packages are looking for funding
    run `npm fund` for details

  found 0 vulnerabilities
● @eFFe0513 ➜/workspaces/cloud-computing-lab/nodejs-app (main) $ cat > server.js << 'EOF'
  const express = require('express');
  const app = express();

  app.get('/', (req, res) => {
    res.json({
      message: 'Hello from Node.js!Federico',
      service: 'nodejs-api'
    });
  });

  app.get('/health', (req, res) => {
    res.json({ status: 'ok' });
  });

  const PORT = process.env.PORT || 3000;
  app.listen(PORT, () => {
    console.log(`✅ Node.js app running on port ${PORT}`);
  });
  EOF
○ @eFFe0513 ➜/workspaces/cloud-computing-lab/nodejs-app (main) $
```

L'utente si sposta in nodejs-app ed esegue npm install express per installare il framework (vengono aggiunti 65 pacchetti). Successivamente, tramite cat > server.js << 'EOF' ... EOF, crea il file server.js con il codice: due endpoint (/ che restituisce un messaggio personalizzato "Hello from Node.js!Federico" e /health che restituisce {"status":"ok"}) e il server in ascolto sulla porta 3000.

```
@eFFe0513 ➜/workspaces/cloud-computing-lab/nodejs-app (main) $ npm start
Debugger attached.

> nodejs-app@1.0.0 start
> node server.js

Debugger attached.
✅ Node.js app running on port 3000
```

Viene lanciato il comando npm start (definito nel package.json come node server.js). Il terminale mostra l'output Debugger attached (dovuto al debugger di VS Code) e il messaggio Node.js app running on port 3000, indicando che il server Express è partito correttamente e rimane in esecuzione in attesa di richieste.

```
@eFFe0513 ➜/workspaces/cloud-computing-lab/nodejs-app (main) $
curl http://localhost:3000/
{"message":"Hello from Node.js!Federico","service":"nodejs-api"}
@eFFe
@eFFe0513 ➜/workspaces/cloud-computing-lab/nodejs-app (main) $
curl http://localhost:3000/health
{"status":"ok"}@eFFe0513 ➜/workspaces/cloud-computing-lab/nodej
s-app (main) $
```

Aprendo un nuovo terminale ed eseguendo curl http://localhost:3000/, si ottiene la risposta JSON {"message":"Hello from Node.js!Federico","service":"nodejs-api"}. Successivamente testando l'endpoint di health con curl http://localhost:3000/health, si riceve {"status":"ok"}. Entrambe le chiamate hanno successo, verificando il corretto funzionamento dell'applicazione all'interno del Dev Container.

Procedimento e Risultati

L'esercitazione è iniziata con la creazione manuale di un Dev Container. Dopo aver preparato una cartella locale e inizializzato un repository Git, è stato realizzato il file .devcontainer/devcontainer.json che specifica un'immagine base Debian, l'installazione di Node.js 24 e Docker-in-Docker, e l'inoltro automatico delle porte 3000, 8080 e 8888. Nel terminale integrato vs locale è apparsa la conferma delle versioni: Node.js v24.15.0 e Docker 29.4.1, e le porte erano già inoltrate su localhost. Si è poi passati a un repository già configurato, cloud-computing-lab, dove sono state verificate le medesime condizioni. All'interno della cartella nodejs-app è stato

installato Express e creato un semplice server.js con due endpoint, uno sulla radice che restituisce un messaggio personalizzato e uno su /health che restituisce {"status":"ok"}. L'avvio con npm start ha fatto partire il server sulla porta 3000 e i test con curl hanno dato esito positivo, confermando il pieno funzionamento dell'applicazione all'interno del Dev Container.

Considerazioni Finali

Questa esercitazione ha mostrato concretamente i vantaggi dei Dev Container in GitHub Codespaces. Configurare manualmente il container ha richiesto pochi passaggi ma ha offerto un controllo completo sull'ambiente, mentre l'uso di un repository preconfigurato ha dimostrato la riproducibilità immediata. L'inoltro automatico delle porte e la presenza di Docker-in-Docker semplificano notevolmente lo sviluppo e il test di applicazioni. In conclusione, i Dev Container rappresentano uno strumento efficace per il cloud computing, garantendo ambienti isolati e pronti all'uso, riducendo i problemi di compatibilità e facilitando il lavoro collaborativo.